Лабораторная работа №6

Реализация основных моделей в подходе сетей Петри

Дагделен Зейнап Реджеповна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**
3.1 Модель SIR 7
3.2 Сети Петри 8
3.3 Используемые численные методы 9

4 Выполнение лабораторной работы **10**
4.1 Базовый прогон модели SIR 12
4.2 Сканирование параметра β в модели SIR 14
4.3 Анимация детерминированной динамики модели SIR 16
4.4 Итоговый отчёт по модели SIR 18

5 Вывод **21**

Список литературы **22**

Список иллюстраций

4.1 Запуск кода 10
4.2 Запуск кода 10
4.3 Компиляция sirpetri_report.jl 11
4.4 Компиляция sirpetri_animate.jl 11
4.5 Компиляция sirpetri_scan_parameters.jl 11
4.6 Компиляция sirpetri_run.jl 12

Список таблиц

1 Цель работы

Освоить моделирование эпидемии SIR с помощью сетей Петри в Julia, реализовать детерминированный и стохастический подходы, провести анализ чувствительности модели, а также преобразовать код в литературный стиль с генерацией исполняемых файлов (Jupyter Notebook, Quarto).

2 Задание

1. Реализовать модель SIR с использованием размеченной сети Петри.
2. Выполнить базовый расчёт (детерминированный и стохастический) с параметрами $\beta = 0.3$, $\gamma = 0.1$.
3. Провести сканирование параметра $\beta\,(0.1 : 0.05 : 0.8)$ и построить графики $\text{peak}_I(\beta)$ и $\text{final}_R(\beta)$.
4. Создать анимацию динамики S, I, R.
5. Преобразовать код в литературный стиль и сгенерировать:
   - чистый код (`.jl`);
   - Jupyter Notebook (`.ipynb`);
   - документацию Quarto (`.qmd`).
6. Выполнить расчёты для дополнительного набора параметров (например, $\beta = 0.5$, $\gamma = 0.05$) и включить результаты в отчёт.

Ниже приведён структурированный текст для отчёта по лабораторной работе на основе предоставленного файла `lab06.pdf`. Текст включает **цель**, **задание** (адаптированное и дополненное для ясности) и **теоретическое введение** (описание модели SIR и сетей Петри).

3 Теоретическое введение

3.1 Модель SIR

Модель SIR является классической эпидемиологической моделью, описывающей распространение инфекционного заболевания в популяции. Вся популяция делится на три группы:

- **S (Susceptible)** – восприимчивые к инфекции особи, которые могут заразиться при контакте с инфицированными.
- **I (Infectious)** – инфицированные особи, способные передавать инфекцию восприимчивым и впоследствии выздоравливать.
- **R (Recovered)** – выздоровевшие особи, приобретающие иммунитет и не участвующие в дальнейшем распространении болезни.

Взаимодействия между группами описываются двумя процессами:

- **Заражение**: $S + I \xrightarrow{\beta} I + I$ – при контакте восприимчивого с инфицированным первый переходит в группу I. Скорость процесса пропорциональна произведению $S \cdot I$ (закон действующих масс) с коэффициентом β (эффективность заражения).
- **Выздоровление**: $I \xrightarrow{\gamma} R$ – инфицированный переходит в группу R с постоянной скоростью γ (обратная величина среднего времени болезни).

3.1.1 Детерминированная форма (ОДУ)

В предположении бесконечно большой популяции динамика концентраций описывается системой обыкновенных дифференциальных уравнений:

$$\begin{cases} \frac{dS}{dt} = -\beta SI \\ \frac{dI}{dt} = \beta SI - \gamma I \\ \frac{dR}{dt} = \gamma I \end{cases}$$

3.1.2 Стохастическая форма

При малой численности популяции существенны случайные флуктуации. Стохастическое моделирование использует **прямой метод Гиллеспи** (SSA – Stochastic Simulation Algorithm). На каждом шаге:

- Вычисляются пропускные способности (hazard functions): $a_{\text{inf}} = \beta SI$, $a_{\text{rec}} = \gamma I$.
- Генерируется время до следующего события: $\Delta t = -\ln(r_1)/(a_{\text{inf}} + a_{\text{rec}})$, где $r_1 \sim \text{Uniform}(0, 1)$.
- С вероятностью $a_{\text{inf}}/(a_{\text{inf}} + a_{\text{rec}})$ происходит заражение, иначе – выздоровление.
- Состояние обновляется дискретно, и процесс повторяется.

3.2 Сети Петри

Сеть Петри – это графовая модель параллельных и асинхронных систем, состоящая из:

- **Позиций (places)** – круги, представляющие состояния (в модели SIR: S, I, R).

- **Переходов (transitions)** – прямоугольники, представляющие события (infection, recovery).
- **Дуг (arcs)** – связи от позиций к переходам и обратно, указывающие, сколько фишек изымается или помещается.
- **Фишек (tokens)** – дискретные единицы, распределённые по позициям.

В данной работе используется **размеченная сеть Петри (Labelled Petri Net)**, где переходы имеют метки и связаны с динамическими законами. На основе такой сети автоматически строится система ОДУ (через закон действующих масс) или запускается алгоритм Гиллеспи. Это позволяет унифицировать описание модели для детерминированного и стохастического режимов.

3.3 Используемые численные методы

- **Tsit5** – метод Рунге-Кутты 5-го порядка (реализация из пакета `OrdinaryDiffEq`) для детерминированного интегрирования.
- **Прямой метод Гиллеспи** (Gillespie SSA) – точный алгоритм для стохастического моделирования химических и биологических реакций.

Такой подход демонстрирует переход от макроскопической (гладкой) динамики к микроскопической (флуктуирующей) при уменьшении масштаба системы, что особенно важно для понимания поведения эпидемий в малых популяциях.

4 Выполнение лабораторной работы

Создала необходимые файлы, куда скопировала весь код, предоставленный в лабораторной работе.

Запустила их всех, сначала пробежав `install_packages.jl`, установив необходимые библиотеки (рис. 4.1, рис. 4.2).

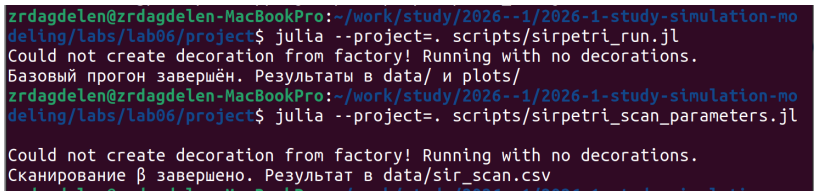

Рисунок 4.1: Запуск кода

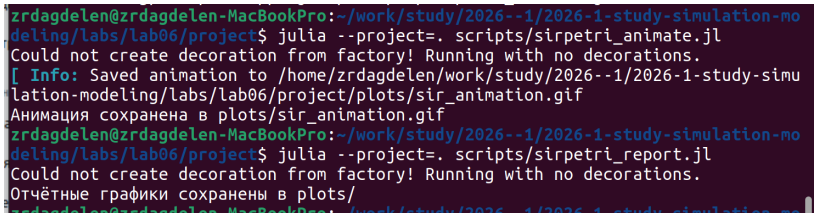

Рисунок 4.2: Запуск кода

Далее создала литературный код для всех основных файлов. После скомпилировала чистый код, jupiter notebook и quarto с помощью файла с кодом `tangle.jl` (рис. 4.3, рис. 4.4, рис. 4.5, рис. 4.6).

```
zrdagdelen@zrdagdelen-MacBookPro:~/work/study/2026--1/2026-1-study-simulation-mo
deling/labs/lab06/project$ julia --project=. scripts/tangle.jl scripts/sirpetri_
report.jl
Генерация из: scripts/sirpetri_report.jl
[ Info: generating plain script file from `~/work/study/2026--1/2026-1-study-sim
ulation-modeling/labs/lab06/project/scripts/sirpetri_report.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling
/labs/lab06/project/scripts/sirpetri_report/sirpetri_report.jl`
 ✓ Чистый скрипт: /home/zrdagdelen/work/study/2026--1/2026-1-study-simulation-mo
deling/labs/lab06/project/scripts/sirpetri_report/sirpetri_report.jl
[ Info: generating markdown page from `~/work/study/2026--1/2026-1-study-simulat
ion-modeling/labs/lab06/project/scripts/sirpetri_report.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling
/labs/lab06/project/markdown/sirpetri_report/sirpetri_report.qmd`
```

Рисунок 4.3: Компиляция sirpetri_report.jl

```
zrdagdelen@zrdagdelen-MacBookPro:~/work/study/2026--1/2026-1-study-simulation-mo
deling/labs/lab06/project$ julia --project=. scripts/tangle.jl scripts/sirpetri_
animate.jl
Генерация из: scripts/sirpetri_animate.jl
[ Info: generating plain script file from `~/work/study/2026--1/2026-1-study-sim
ulation-modeling/labs/lab06/project/scripts/sirpetri_animate.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling
/labs/lab06/project/scripts/sirpetri_animate/sirpetri_animate.jl`
 ✓ Чистый скрипт: /home/zrdagdelen/work/study/2026--1/2026-1-study-simulation-mo
deling/labs/lab06/project/scripts/sirpetri_animate/sirpetri_animate.jl
[ Info: generating markdown page from `~/work/study/2026--1/2026-1-study-simulat
ion-modeling/labs/lab06/project/scripts/sirpetri_animate.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling
/labs/lab06/project/markdown/sirpetri_animate/sirpetri_animate.qmd`
```

Рисунок 4.4: Компиляция sirpetri_animate.jl

```
zrdagdelen@zrdagdelen-MacBookPro:~/work/study/2026--1/2026-1-study-simulation-mo
deling/labs/lab06/project$ julia --project=. scripts/tangle.jl scripts/sirpetri_
scan_parameters.jl
Генерация из: scripts/sirpetri_scan_parameters.jl
[ Info: generating plain script file from `~/work/study/2026--1/2026-1-study-sim
ulation-modeling/labs/lab06/project/scripts/sirpetri_scan_parameters.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling
/labs/lab06/project/scripts/sirpetri_scan_parameters/sirpetri_scan_parameters.jl
`
 ✓ Чистый скрипт: /home/zrdagdelen/work/study/2026--1/2026-1-study-simulation-mo
deling/labs/lab06/project/scripts/sirpetri_scan_parameters/sirpetri_scan_paramet
ers.jl
[ Info: generating markdown page from `~/work/study/2026--1/2026-1-study-simulat
ion-modeling/labs/lab06/project/scripts/sirpetri_scan_parameters.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling
/labs/lab06/project/markdown/sirpetri_scan_parameters/sirpetri_scan_parameters.q
md`
```

Рисунок 4.5: Компиляция sirpetri_scan_parameters.jl

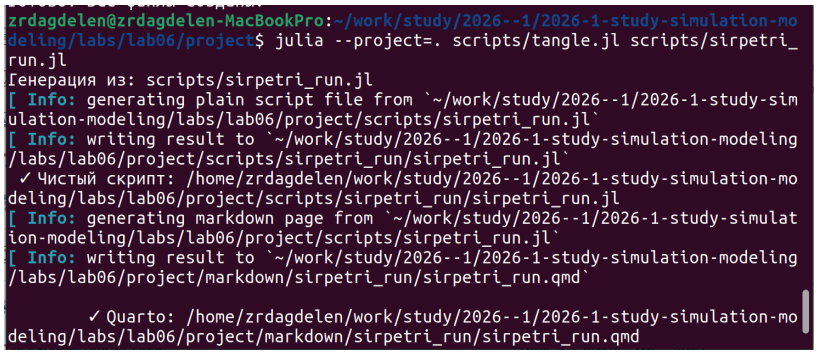

Рисунок 4.6: Компиляция sirpetri_run.jl

Кад из файлов и анализ результатов предоставлен ниже.

4.1 Базовый прогон модели SIR

4.1.1 Подготовка окружения

Подключаем необходимые пакеты для работы с моделью SIR.

```
using DrWatson
@quickactivate "project"
using Random
include(srcdir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

4.1.2 Параметры модели

Задаём основные параметры эпидемии:

- $/beta$ (бета) — коэффициент заражения (скорость перехода S -> I)
- $/gamma$ (гамма) — коэффициент выздоровления (скорость перехода I -> R)

- `tmax` — максимальное время симуляции

```
β = 0.3
γ = 0.1
tmax = 100.0
```

4.1.3 Создание сети Петри

Строим размеченную сеть Петри для модели SIR. Начальная маркировка: 990 восприимчивых, 10 инфицированных, 0 выздоровевших.

```
net, u0, states = build_sir_network(β, γ)
```

4.1.4 Детерминированная симуляция

Запускаем детерминированную симуляцию (решение системы ОДУ). Результаты сохраняются в CSV-файл.

```
df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [β,
CSV.write(datadir("sir_det.csv"), df_det)
```

4.1.5 Стохастическая симуляция

Запускаем стохастическую симуляцию (алгоритм Гиллеспи). Устанавливаем seed для воспроизводимости результатов.

```
Random.seed!(123)
df_stoch = simulate_stochastic(net, u0, (0.0, tmax), rates = [β, γ])
CSV.write(datadir("sir_stoch.csv"), df_stoch)
```

4.1.6 Визуализация результатов

Строим графики динамики S, I, R для обоих типов симуляции.

```
p_det = plot_sir(df_det)
savefig(plotsdir("sir_det_dynamics.png"))

p_stoch = plot_sir(df_stoch)
savefig(plotsdir("sir_stoch_dynamics.png"))
```

4.1.7 Завершение

```
println("Базовый прогон завершён. Результаты в data/ и plots/")
```

4.2 Сканирование параметра β в модели SIR

4.2.1 Подготовка окружения

Подключаем необходимые пакеты и модули.

```
using DrWatson
@quickactivate "project"
include(srcdir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

4.2.2 Параметры сканирования

Задаём диапазон значений коэффициента заражения β и фиксированные параметры.

- β_range — диапазон значений β от 0.1 до 0.8 с шагом 0.05
- γ_fixed — фиксированный коэффициент выздоровления
- tmax — максимальное время симуляции

```
β_range = 0.1:0.05:0.8
γ_fixed = 0.1
tmax = 100.0
```

4.2.3 Цикл сканирования

Для каждого значения β запускаем детерминированную симуляцию и записываем: - peak_I — максимальное число инфицированных (пик эпидемии) - final_R — конечное число выздоровевших

```
results = []
for β in β_range
    net, u0, _ = build_sir_network(β, γ_fixed)
    df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [β,
    peak_I = maximum(df.I)
    final_R = df.R[end]
    push!(results, (β = β, peak_I = peak_I, final_R = final_R))
end
```

4.2.4 Сохранение результатов

Преобразуем результаты в DataFrame и сохраняем в CSV-файл.

```
df_scan = DataFrame(results)
CSV.write(datadir("sir_scan.csv"), df_scan)
```

4.2.5 Визуализация

Строим график зависимости пика эпидемии (Peak I) и конечного числа выздоровевших (Final R) от коэффициента заражения β.

```
p = plot(
    df_scan.β,
    [df_scan.peak_I df_scan.final_R],
    label = ["Peak I" "Final R"],
    marker = :circle,
    xlabel = "β (infection rate)",
    ylabel = "Population",
)
savefig(plotsdir("sir_scan.png"))
```

4.2.6 Завершение

```
println("Сканирование β завершено. Результат в data/sir_scan.csv")
```

4.3 Анимация детерминированной динамики модели SIR

4.3.1 Подготовка окружения

Подключаем необходимые пакеты и модули.

```
using DrWatson
@quickactivate "project"
include(srcdir("SIRPetri.jl"))
```

```
using .SIRPetri
using Plots
```

4.3.2 Параметры модели

Задаём параметры для анимации: - `β = 0.3` — коэффициент заражения - `γ = 0.1` — коэффициент выздоровления - `tmax = 100.0` — максимальное время симуляции

```
β = 0.3
γ = 0.1
tmax = 100.0
```

4.3.3 Запуск симуляции

Создаём сеть Петри и запускаем детерминированную симуляцию с шагом сохранения 0.5 для плавности анимации.

```
net, u0, _ = build_sir_network(β, γ)
df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [β, γ])
```

4.3.4 Создание анимации

Создаём анимацию, где каждый кадр показывает столбчатую диаграмму количества людей в каждой группе (S, I, R) в соответствующий момент времени. Берём каждый 5-й кадр для оптимальной плавности.

```
anim = Animation()
for i in 1:5:length(df.time)
    p = bar(
```

```
        ["S", "I", "R"],
        [df.S[i], df.I[i], df.R[i]],
        ylims = (0, 1000),
        title = "Time = $(round(df.time[i], digits=1))",
        ylabel = "Population",
        label = "",
        color = [:green, :red, :blue]
    )
    frame(anim, p)
end
```

4.3.5 Сохранение анимации

Сохраняем анимацию в GIF-файл с частотой 5 кадров в секунду.

```
gif(anim, plotsdir("sir_animation.gif"), fps = 5)
```

4.3.6 Завершение

```
println("Анимация сохранена в plots/sir_animation.gif")
```

4.4 Итоговый отчёт по модели SIR

4.4.1 Загрузка данных

Загружаем ранее сохранённые результаты симуляций.

```julia
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
```

Загрузка данных из CSV-файлов: - `sir_det.csv` — детерминированная симуляция - `sir_stoch.csv` — стохастическая симуляция - `sir_scan.csv` — результаты сканирования параметра β

```julia
df_det = CSV.read(datadir("sir_det.csv"), DataFrame)
df_stoch = CSV.read(datadir("sir_stoch.csv"), DataFrame)
df_scan = CSV.read(datadir("sir_scan.csv"), DataFrame)
```

4.4.2 Сравнение детерминированной и стохастической динамики

Строим сравнительный график числа инфицированных I(t) для детерминированного и стохастического случаев.

```julia
n_points = min(length(df_det.time), length(df_stoch.time))
p1 = plot(
    df_det.time[1:n_points],
    [df_det.I[1:n_points] df_stoch.I[1:n_points]],
    label = ["Deterministic I" "Stochastic I"],
    xlabel = "Time",
    ylabel = "Infected",
    title = "Comparison of Deterministic and Stochastic Dynamics",
    linewidth = 2,
)
savefig(plotsdir("comparison.png"))
```

4.4.3 Анализ чувствительности к параметру β

Строим график зависимости пика эпидемии (максимального числа инфицированных) от коэффициента заражения β.

```
p2 = plot(
    df_scan.β,
    df_scan.peak_I,
    marker = :circle,
    linewidth = 2,
    xlabel = "β (infection rate)",
    ylabel = "Peak Infected",
    title = "Sensitivity Analysis: Peak I vs β",
)
savefig(plotsdir("sensitivity.png"))
```

4.4.4 Завершение

```
println("Отчётные графики сохранены в plots/")
```

5 Вывод

Реализована модель SIR с использованием сетей Петри в детерминированном и стохастическом вариантах. Проведён анализ чувствительности к параметру β. Метод литературного программирования позволил автоматически генерировать исполняемые файлы и отчёты.

Список литературы

- Лабораторная №6